Here is a table of **most commonly used Day-to-Day and One-Time Shell Scripting Commands** with explanations, tailored for **Cloud, DevOps, and Linux** job roles:

Shell Scripting Commands for Cloud/DevOps/Linux Roles

Category	Command / Script Element	Day-to-Day / One-Time	Purpose / Explanation
Variables	VAR=valueecho $VAR	Day-to-Day	Define and use variables. Used in all scripts for storing values.
Input/Output	read varname	Day-to-Day	Read user input into a variable.
	echo "message"	Day-to-Day	Print output to terminal or logs.
Conditionals	if [$a -eq $b]; then ... fi	Day-to-Day	Execute code conditionally. Common in automation scripts.
	case $var in ...)	Day-to-Day	Multi-condition checks. Useful in menus, status checking.
Loops	for i in ...; do ... done	Day-to-Day	Iterate over items or files.
	while [condition]; do ... done	Day-to-Day	Repeat code while condition is true.
Functions	function_name () { ... }	Day-to-Day	Define reusable code blocks in scripts.
Comments	# This is a comment	Day-to-Day	Add notes/explanations in scripts.
Exit Status	exit 0, exit 1	Day-to-Day	Indicate script success or failure.
	$?	Day-to-Day	Check exit status of last command.
Command Subst.	VAR=$(command)	Day-to-Day	Store output of a command in a variable.
Logging	>> logfile.log	Day-to-Day	Append output to log file.
	exec > logfile 2>&1	One-Time	Redirect all stdout/stderr to a log (used in automation).
File Ops	if [-f file]if [-d dir]	Day-to-Day	Check if file or directory exists.

	rm -rf $dir	One-Time	Remove directory recursively (handle with caution).
Script Headers	#!/bin/bash	One-Time	Shebang line to define script interpreter.
Scheduling	crontab -e	One-Time	Schedule shell scripts for auto-run (e.g., backups, sync).
Script Arguments	$1, $2, $@, $#	Day-to-Day	Access arguments passed to a script.
Exit on Error	set -e	One-Time / Secure Coding	Exit script if any command fails.
Debugging	set -x	One-Time (Debugging)	Print each command before executing it (for debugging).
Arrays	arr=(val1 val2)echo ${arr[0]}	Day-to-Day	Use arrays to handle multiple items like file lists.
Networking	ping, curl, wget	Day-to-Day	Test server connectivity or API endpoints.
Package Checks	dpkg -l, rpm -qa, which	Day-to-Day	Check if software is installed in shell scripts.
Loop with Files	for file in *.log; do ... done	Day-to-Day	Process multiple files in automation (e.g., log rotation).
Permissions	chmod +x script.sh	One-Time	Make a script executable.
Archiving/Backup	tar -czf backup.tar.gz /dir	Day-to-Day	Archive files/directories for backup in automation.
Remote Execution	ssh user@host "cmd"	Day-to-Day	Run scripts/commands on remote servers (DevOps routine).
Monitoring	tail -f logfile	Day-to-Day	Monitor live logs in shell for debugging.
Text Processing	grep, awk, sed, cut	Day-to-Day	Parse and process text/log files in scripts.
Process Mgmt	`ps aux	grep process`	Day-to-Day	

 Best Practices for Shell Scripts

- Use **set -euo pipefail** at the top for robust scripts.
- Always validate input with condition checks.
- Add **logging and error handling**.
- Use **functions** for modularity.
- Test scripts in a **staging environment** before production.

Would you like a downloadable .sh template for CI/CD, AWS automation, or Linux admin tasks?

Here's a collection of **.sh script templates** for **CI/CD**, **AWS automation**, and **Linux administration** tasks. These templates can be used as building blocks for automation in Cloud, DevOps, and Linux environments.

---

1. **CI/CD Build & Deploy Script Template (**deploy.sh**)**

```
#!/bin/bash
set -euo pipefail

# Variables
APP_NAME="myapp"
BUILD_DIR="./build"
S3_BUCKET="my-ci-artifacts"
DEPLOY_ENV="prod"

echo "[INFO] Starting CI/CD pipeline for $APP_NAME..."

# Build Step
echo "[INFO] Building the application..."
npm install
npm run build

# Upload artifacts to S3
echo "[INFO] Uploading artifacts to S3 bucket: $S3_BUCKET"
aws s3 cp "$BUILD_DIR" "s3://$S3_BUCKET/$APP_NAME/$DEPLOY_ENV/" --recursive

# Notify
echo "[SUCCESS] Deployment artifacts uploaded successfully!"

# Optional: Trigger deployment via AWS CodeDeploy, Lambda, etc.
```

---

2. AWS EC2 Instance Automation Script (setup-ec2.sh)

```
#!/bin/bash
set -euo pipefail

# Variables
PACKAGES=("git" "nginx" "docker.io")
EC2_USER="ec2-user"
LOG_FILE="/var/log/ec2-setup.log"

echo "[INFO] Starting EC2 setup..." | tee -a $LOG_FILE

# Update packages
sudo apt update -y && sudo apt upgrade -y

# Install packages
for pkg in "${PACKAGES[@]}"; do
  echo "[INFO] Installing $pkg..." | tee -a $LOG_FILE
  sudo apt install -y $pkg
done

# Start and enable services
sudo systemctl enable nginx
sudo systemctl start nginx

echo "[SUCCESS] EC2 setup completed." | tee -a $LOG_FILE
```

---

3. Linux Backup and Log Rotation Script (daily-backup.sh)

```
#!/bin/bash
set -euo pipefail

BACKUP_DIR="/backup"
SRC_DIR="/var/www/html"
LOG_FILE="/var/log/backup.log"
DATE=$(date +%Y-%m-%d)
TAR_FILE="$BACKUP_DIR/site-backup-$DATE.tar.gz"

# Create backup
echo "[INFO] Creating backup for $SRC_DIR on $DATE..." | tee -a $LOG_FILE
tar -czf "$TAR_FILE" "$SRC_DIR"

# Optional: Remove backups older than 7 days
find "$BACKUP_DIR" -type f -mtime +7 -name "*.tar.gz" -exec rm -f {} \;

echo "[SUCCESS] Backup complete: $TAR_FILE" | tee -a $LOG_FILE
```

---

4. AWS Lambda Deployment Script (deploy-lambda.sh)

```
#!/bin/bash
set -euo pipefail

FUNCTION_NAME="my-lambda-function"
ZIP_FILE="function.zip"
HANDLER="index.handler"
ROLE_ARN="arn:aws:iam::123456789012:role/lambda-ex"

# Zip the function
echo "[INFO] Zipping Lambda function..."
zip -r "$ZIP_FILE" .

# Deploy using AWS CLI
aws lambda update-function-code \
  --function-name "$FUNCTION_NAME" \
  --zip-file "fileb://$ZIP_FILE"

echo "[SUCCESS] Lambda function updated successfully!"
```

---

5. User Account and SSH Setup Script (user-setup.sh)

```
#!/bin/bash
set -euo pipefail

USERNAME="devuser"
SSH_DIR="/home/$USERNAME/.ssh"
AUTHORIZED_KEYS="$SSH_DIR/authorized_keys"
PUB_KEY="ssh-rsa AAAAB3..."

# Create user
sudo useradd -m -s /bin/bash $USERNAME

# Setup SSH
sudo mkdir -p $SSH_DIR
sudo touch $AUTHORIZED_KEYS
echo "$PUB_KEY" | sudo tee -a $AUTHORIZED_KEYS > /dev/null
sudo chmod 700 $SSH_DIR
sudo chmod 600 $AUTHORIZED_KEYS
sudo chown -R $USERNAME:$USERNAME $SSH_DIR

echo "[SUCCESS] User $USERNAME created and SSH configured."
```

Want to Bundle These into a .zip?

Let me know if you'd like me to package all of these .sh files into a downloadable .zip or GitHub-ready repository format.

Would you like to:

- Download all as one .zip?
- Customize for Jenkins, GitHub Actions, or AWS CodePipeline?
- Add environment variable handling or .env integration?